

Google Cloud IAM & Administration

Internal Technical Documentation — Production-Ready Learning Guide

Learning approach

Follow the same progression used in the Managed Airflow documentation: understand the purpose first, then the architecture and core concepts, then the console workflow, security controls, monitoring, troubleshooting, production design and operational checklist.

1. Introduction

Google Cloud Identity and Access Management (IAM) controls who can access Google Cloud resources and what they are allowed to do. IAM is an authorization system: authentication establishes who or what is making a request, while IAM evaluates whether that principal has the required permissions.

- Users, groups, service accounts and other supported principals can receive access.
- Permissions are grouped into roles.
- Policies bind principals to roles at resource scopes.

2. Why IAM is needed

Without IAM, cloud resources cannot be governed safely at scale. IAM lets organizations separate responsibilities, restrict sensitive operations and provide applications with controlled identities.

- Least privilege: grant only the permissions required.
- Separation of duties: avoid giving one identity unnecessary control over unrelated resources.
- Central governance: apply policies at organization, folder, project and resource levels.
- Auditability: access changes and activity can be investigated using audit logs.

3. Resource hierarchy and policy inheritance

Google Cloud resources are organized in a hierarchy. A simplified model is organization → folders → projects → resources. Policies can be inherited by descendants depending on the policy type and resource.

- Use organization/folder-level controls for broad governance.
- Use project/resource-level roles for workload-specific access.
- Always check the effective access at the target resource before changing permissions.

4. Core IAM terminology

Term	Meaning
Principal	Identity receiving access, such as a user, group, service account or supported workload identity.
Permission	A specific allowed operation on a resource.
Role	A collection of permissions.
IAM policy	Bindings that associate principals with roles and may include conditions.
Resource	The Google Cloud object being protected, such as a project, bucket, dataset or service.
Service account	A workload identity used by applications and Google Cloud services.
Least privilege	Grant the smallest practical set of permissions needed for a task.

5. Principals: users, groups and service accounts

Users are normally human identities. Groups simplify access management for teams. Service accounts represent workloads and applications.

- Prefer groups for team-based access rather than granting many individual users the same role.

- Use service accounts for application workloads.
- Avoid using personal user identities as application credentials.

6. Roles and permission design

Roles determine which permissions a principal receives. Google Cloud provides predefined roles and supports custom roles for organizations that need narrower control.

- Prefer predefined roles when they satisfy the requirement.
- Use custom roles when a well-defined least-privilege requirement cannot be met by predefined roles.
- Avoid primitive broad roles such as Owner/Editor for production workloads unless there is a specific governance reason.

7. IAM Conditions and contextual access

IAM Conditions can make an access binding conditional on attributes such as resource context or time. Conditions should be documented and tested carefully because a technically correct role can still fail when its condition is not satisfied.

- Use conditions when they materially improve security or governance.
- Test both allowed and denied paths.
- Document why a condition exists and who owns it.

8. Service accounts and workload identity

A production workload should receive an explicit service identity with only the permissions it needs. Avoid long-lived service-account keys when a more secure identity mechanism is available.

- Use workload identity mechanisms supported by the runtime/platform where possible.
- If keys are unavoidable, protect, rotate and monitor them according to organizational policy.
- Separate service accounts by workload/environment when practical.

9. Encryption, secrets and administrative separation

IAM controls authorization; it does not replace application-level secret management or encryption controls. Use Secret Manager for application secrets and Cloud KMS when customer-managed keys are required.

- Do not place passwords, API keys or service-account private keys in source code or documentation.
- Separate developers, deployers and runtime identities where appropriate.
- Review who can administer IAM itself because IAM changes can grant broad access.

10. Cloud Console administration workflow

A safe administration workflow is: select the correct project/resource → inspect current access → identify the exact principal → choose the narrowest suitable role → add a condition if required → apply the change → verify effective access.

- Double-check the project before changing access.
- Prefer the smallest resource scope that meets the requirement.
- Record the reason for elevated access.
- Remove temporary access when it is no longer needed.

11. Monitoring and auditability

IAM administration should be observable. Google Cloud audit logs and policy views help determine who changed access and whether an identity performed a sensitive operation.

- Review audit logs for important administrative changes.
- Monitor unusual privilege changes.

- Use Cloud Logging/Monitoring integrations appropriate to the organization.
- Maintain an owner for production IAM policies.

12. Troubleshooting IAM failures

A permission-denied error should be investigated systematically rather than solved by granting a broad role.

- Identify the principal actually making the request.
- Identify the exact resource being accessed.
- Identify the required permission from the service documentation/error.
- Check inherited and direct role bindings.
- Check IAM Conditions or deny policies if applicable.
- Grant the narrowest missing access and retest.

13. Production access model

A production IAM design should distinguish human access, deployment access and runtime access.

- Humans: groups and role-based access.
- CI/CD: dedicated deployment identity with only deployment permissions.
- Runtime: service account/workload identity with data-plane permissions only.
- Break-glass access: tightly controlled, monitored and documented.

14. Common beginner mistakes

- Granting Owner/Editor to make an error disappear.
- Using personal credentials inside applications.
- Creating long-lived service-account keys unnecessarily.
- Changing IAM in the wrong project.
- Forgetting inherited access.
- Not documenting temporary elevated access.
- Confusing authentication with authorization.

15. End-to-end administration scenario

Suppose a Dataflow pipeline must read Pub/Sub, write BigQuery and use Cloud Storage for staging. A good design identifies the Dataflow worker identity, grants only the permissions needed on those resources, validates network access separately and monitors the resulting job. The same pattern applies to Cloud Functions, Dataproc, Composer, Spanner and other services.

- Start from the workload action, not from a role name.
- Map action → required permission → narrowest role → narrowest resource scope.
- Test the workload before promoting the configuration to production.

16. Operational checklist

- Correct organization/folder/project selected.
- Human access is group-based where practical.
- Runtime identities are dedicated service accounts/workload identities.
- Roles are least-privilege and resource-scoped.
- Temporary elevated access has an owner and expiry/review process.
- Conditions/deny policies are documented when used.

- Service-account keys are avoided or tightly controlled.
- Secrets are stored outside source code.
- IAM changes are auditable.
- Permission-denied troubleshooting follows a systematic workflow.
- Production IAM changes are reviewed and documented.

17. Final learning outcome

After completing this guide, the learner should be able to explain the Google Cloud resource hierarchy, distinguish principals/permissions/roles/policies, create safe service identities, troubleshoot authorization failures and design least-privilege production access.

Official references

- <https://docs.cloud.google.com/iam/docs/overview>
- <https://docs.cloud.google.com/iam/docs/roles-overview>
- <https://docs.cloud.google.com/iam/docs/service-account-overview>
- <https://docs.cloud.google.com/iam/docs/audit-logging>